

In first approach, we chopped our RAM into variable sized segments (wde, heap, stack etc) but this led to external fragmentation.

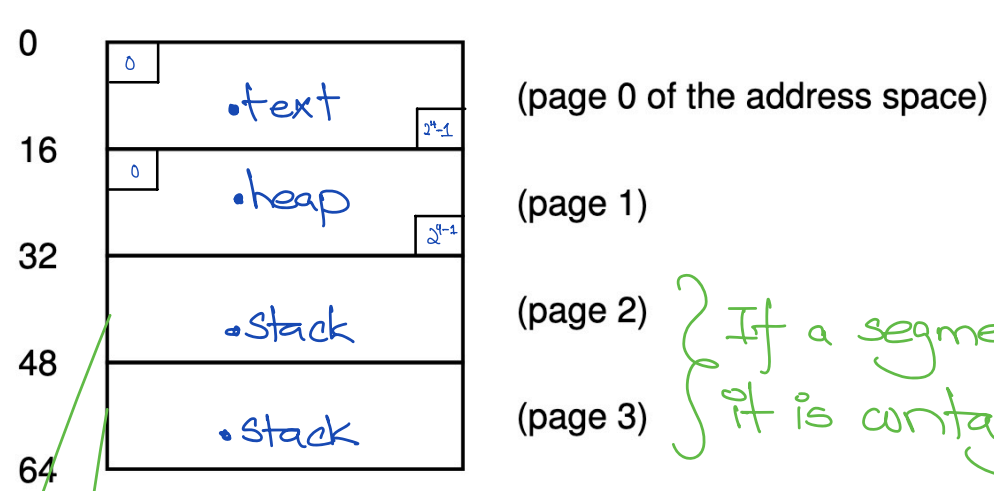
We divide a process' address space into fixed-sized units called pages.

We view physical memory as an array of fixed sized slots called page frames.

Each page frame contains a single page.

THE CRUX:
HOW TO VIRTUALIZE MEMORY WITH PAGES
How can we virtualize memory with pages, so as to avoid the problems of segmentation? What are the basic techniques? How do we make those techniques work well, with minimal space and time overheads?

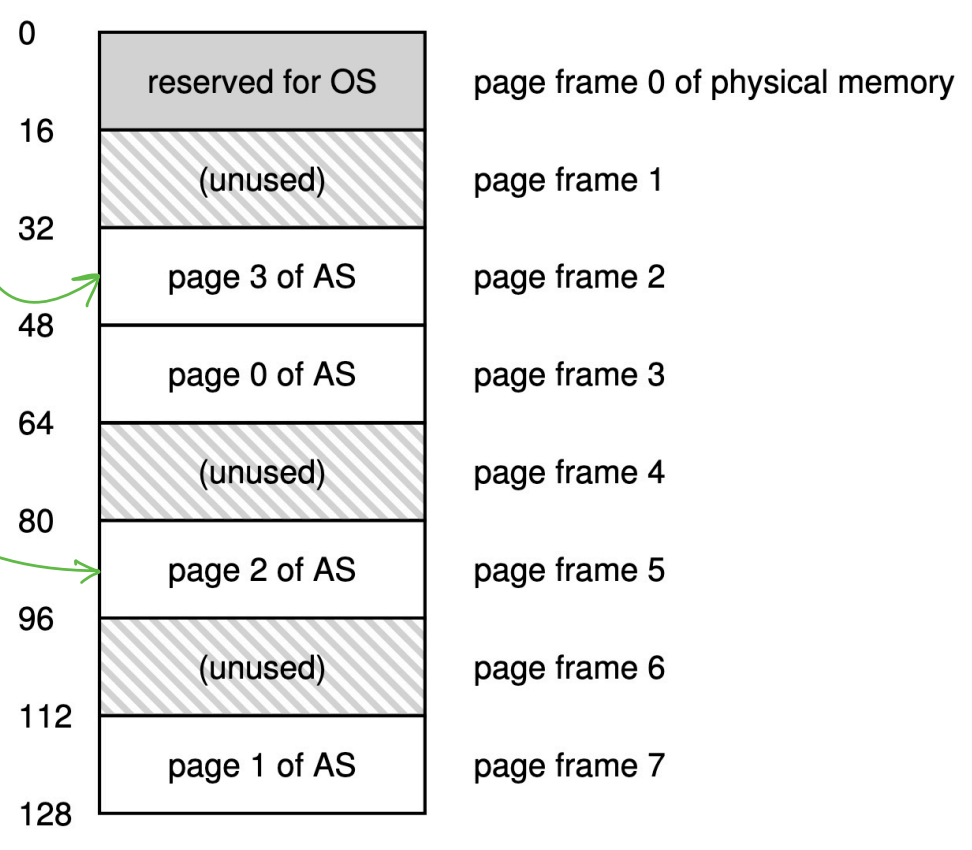
A simple example & Overview



→ virtual address space of 64 bytes with 4 16 bytes pages

If a segment spans two pages, then in VAS it is contiguous but in RAM, it is NOT.

Figure 18.1: A Simple 64-byte Address Space (AS)



→ Physical memory of 128 bytes with 8 16 byte page frames.

Figure 18.2: A 64-Byte Address Space In A 128-Byte Physical Memory

Advantages of this approach

- flexibility

↳ Allows the system to support the abstraction of address-space effectively, regardless how a process uses it.

For example,

we won't make assumptions about the growth direction of stack & heap.

- Simplicity of free-space management:

↳ OS keeps a free-list of free pages

For example,

To place our 64 byte address space in memory, the OS just grabs the first four free pages of the list.

- No external fragmentation

Disadvantages:

↳ Slow (solved in Ch.19)

↳ Takes too much space (solved in Ch.20)

↳ To record where each virtual page (VP) is placed in physical memory, the OS keeps a per-process data structure called page table.

↳ A major role of this is for address translation.

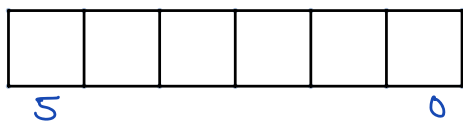
VP	PF
0	3
1	7
2	5
3	2

Address translation example,

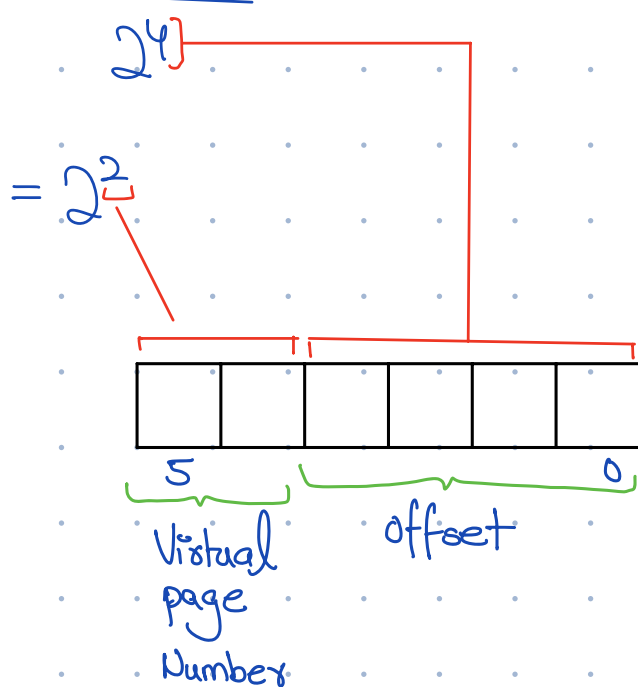
↳ Assume we have instruction:

```
movl <virtual address>, %eax
```

Our virtual address is 6 bits ($2^6 = 64$ bytes).



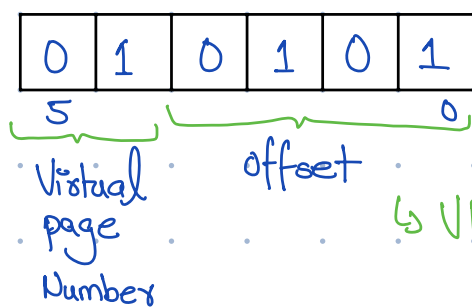
$$\text{VPN bits} = \frac{2^6}{2^4}$$



Assume Virtual address to be 21;

`movl 21, %eax`

In binary 21 is:



VA of 21 is on 0101 offset of VP 01

From the page table $\text{VPN} = 1 \rightarrow \text{PFN} = 7$

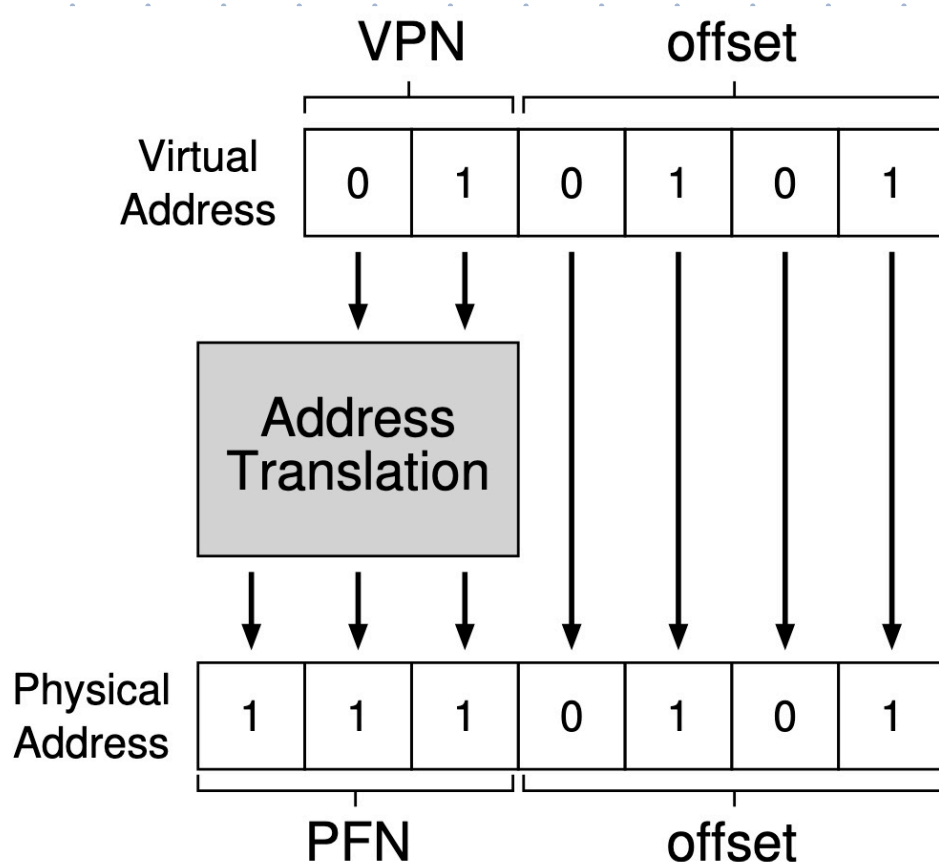


Figure 18.3: The Address Translation Process

Where are pages stored?

- Page tables can get very large.

For example,

let 32-bit address with 4KB page size. This virtual address splits into:

- 20 bit VPN

$$\frac{2^{32}}{2^{12}} = 2^{20}$$

- 12 bit offset

Assume 4 byte per page table entry, we need a page table of size:

$$= \underbrace{2^{20}}_{\text{Total pages}} \times 2^2 = 4\text{MB}$$

For 100 running processes, OS would need 400MB of memory in total to store the 100 per-process page tables.

Whats actually in the page table?

- Page table is a data structure that maps virtual page numbers to page frame numbers.

- The simplest page table is a linear page table i.e. an array.

The OS indexes the array by virtual page number & looks up page table entry at that index i.e. page frame number.

- A Page table entry (PTE) has:

- Valid bit:

- Indicates whether a page occupies space in

physical memory

5.50, when a process tries to access an invalid page, the OS generate a trap.

•code	{ valid
•heap	
	{ Invalid i.e NOT loaded into physical memory
•stack	{ valid

- Protection bits

- ↳ Indicates whether a page could be read from, written to, or executed from.

- Present bit

- ↳ Indicates whether a page is in physical memory or disk.

- Dirty bit:

- Indicating whether a page has been modified since it was brought into memory.

- Reference / Accessed bit.

5. Used to track whether a page has been accessed

- ↳ Useful for determining which pages are popular
- ↳ This knowledge is critical during page replacement.



Figure 18.5: An x86 Page Table Entry (PTE)

Shows an example page table entry from the x86 architecture. It contains a present bit (P); a read/write bit (R/W) which determines if writes are allowed to this page; a user/supervisor bit (U/S) which determines if user-mode processes can access the page; a few bits (PWT, PCD, PAT, and G) that determine how hardware caching works for these pages; an accessed bit (A) and a dirty bit (D); and finally, the page frame number (PFN) itself

Paging: Too Slow

```
1 // Extract the VPN from the virtual address
2 VPN = (VirtualAddress & VPN_MASK) >> SHIFT
3
4 // Form the address of the page-table entry (PTE)
5 PTEAddr = PTBR + (VPN * sizeof(PTE))
6
7 // Fetch the PTE
8 PTE = AccessMemory(PTEAddr)
9
10 // Check if process can access the page
11 if (PTE.Valid == False)
12     RaiseException(SEGMENTATION_FAULT)
13 else if (CanAccess(PTE.ProtectBits) == False)
14     RaiseException(PROTECTION_FAULT)
15 else
16     // Access OK: form physical address and fetch it
17     offset = VirtualAddress & OFFSET_MASK
18     PhysAddr = (PTE.PFN << PFN_SHIFT) | offset
19     Register = AccessMemory(PhysAddr)
```

Figure 18.6: Accessing Memory With Paging

Assume instruction

```
movl 21, %eax
```

VPN = 0x30, SHIFT = 4 (no. of bits in offset)

So,

010101

masking it: 010000

shifting it: 01

Now, Index into PTB (in memory) to get PFN

Concatenate PFN with offset.

Finally,

we have our physical address

In short, we perform 2 memory accesses per memory access:

1. Loading PFN from memory
2. Loading data/instruction itself

A memory trace

Assume code in C:

```
int array[1000];
...
for (i = 0; i < 1000; i++)
    array[i] = 0;
```

In asm:

```
1024 movl $0x0, (%edi, %eax, 4)
1028 incl %eax
1032 cmpl $0x03e8, %eax
1036 jne 1024
```

Explanation below

2
2
2
2

10 Total memory accesses

1024: Move 0 into the virtual address of location of the array.

This address is computed by taking contents of %edi & adding %eax multiplied by 4

↓
holds base/
starting addr
of array

↓
holds array
index

↓
size of
int

1028: increments index held in %eax

1032: Compares index with 1000

1036: If comparison shows two values are NOT yet equal, then jump start of code i.e loop

How many memory accesses?

Assume virtual address space size of 64KB, page size

of 1KB, linear page table located at physical address 1024.

As each page is 1KB (0-1023), 1024 lies on 2nd VP i.e. VP1, which maps VPN 1 $\rightarrow$ PFN 4.

The array is 4000 bytes & resides at virtual addresses 40000 through 44000, it takes 4 pages, which maps as:

VPN 39 $\rightarrow$ PFN 7

VPN 40 $\rightarrow$ PFN 8

VPN 41 $\rightarrow$ PFN 9

VPN 42 $\rightarrow$ PFN 10

When the code runs, each instruction generates two memory references:

1. To page table to find the PFN
2. To instruction itself to fetch it

In addition, there is an explicit memory reference via mov, this adds 2 more memory accesses.

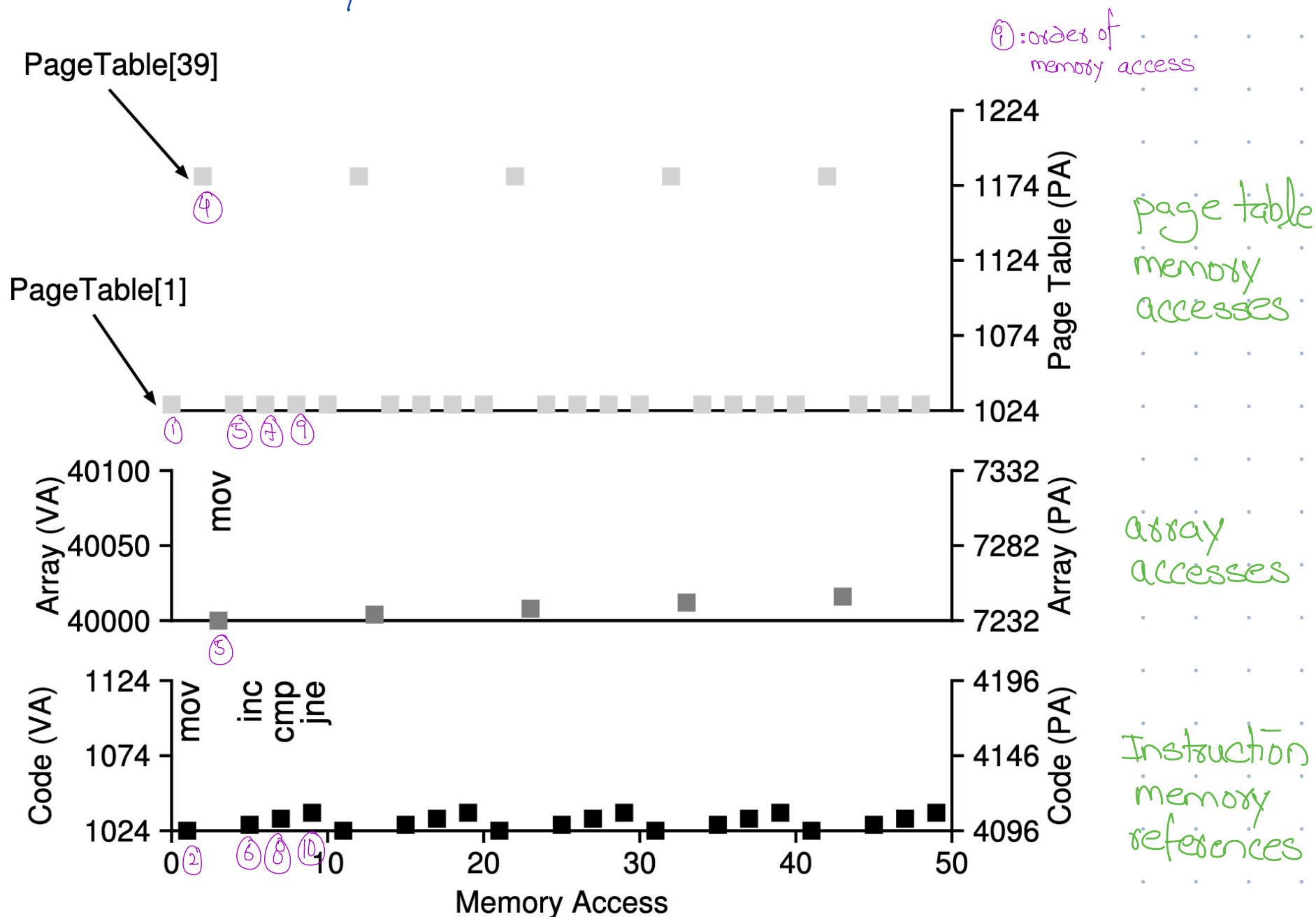


Figure 18.7: A Virtual (And Physical) Memory Trace

Shows memory access for first 5 iterations